Annota

Web Audio Editor

Complete Documentation and Usage Guide

Waad Ben Kheder

April 13, 2026

A fully client-side, browser-based audio editor built with TypeScript and Vite.
All audio editing happens in the browser — no backend, no server-side processing.

Contents

1 Introduction **3**
1.1 Quick Start . . . 3
1.2 Build Commands . . . 3

2 User Interface Overview **4**
2.1 Menu Bar . . . 4
2.2 Toolbar Groups . . . 4

3 Loading Audio **4**
3.1 File Picker . . . 4
3.2 Drag and Drop . . . 5
3.3 Multiple Tracks . . . 5

4 Playback **5**
4.1 Transport Controls . . . 5
4.2 Playback Speed . . . 5
4.3 Seeking . . . 5

5 Visualization **6**
5.1 Waveform . . . 6
5.2 Spectrogram . . . 6
5.3 Zoom and Scroll . . . 6
5.4 Dark / Light Theme . . . 6

6 Selection and Editing **6**
6.1 Creating a Selection . . . 6
6.2 Clipboard Operations . . . 7
6.3 Exporting Audio . . . 7

7 Audio Effects **7**
7.1 Fade In / Fade Out . . . 7
7.2 Normalize . . . 7
7.3 Adjust Gain . . . 7
7.4 Parametric EQ . . . 8
7.5 Compressor / Limiter . . . 8
7.6 Reverb . . . 8
7.7 Noise Reduction . . . 8
7.8 Speed / Pitch Change . . . 8
7.9 Saturation . . . 8
7.10 Resample . . . 9

8 Analysis **9**
8.1 Spectrogram Analysis . . . 9
8.2 Mel Filterbank . . . 9
8.3 MFCC (Mel-Frequency Cepstral Coefficients) . . . 9
8.4 Save Analysis as Image . . . 10

9 Label Track (Annotations) 10
9.1 Label Types . . . 10
9.2 Creating Labels . . . 10
9.3 Editing Labels . . . 10
9.4 Categories . . . 10
9.5 Export Formats . . . 10
9.6 Import . . . 10

10 Per-Track Controls 11
10.1 Volume . . . 11
10.2 Pan . . . 11
10.3 Mute and Solo . . . 11
10.4 Resizable Track Heights . . . 11
10.5 Mixdown Export . . . 11

11 Undo / Redo 11

12 Project Save and Load 12
12.1 Saving . . . 12
12.2 Auto-Save . . . 12
12.3 Loading . . . 12

13 Snap to Grid 12

14 Keyboard Shortcuts 13

15 Technical Architecture 13
15.1 Technology Stack . . . 13
15.2 File Structure . . . 13
15.3 Canvas Architecture . . . 14
15.4 Viewport System . . . 15
15.5 Waveform Mipmap . . . 15
15.6 STFT and Spectrogram . . . 15
15.7 FFT Implementation . . . 16
15.8 Web Worker . . . 16
15.9 Theme System . . . 16
15.10 Undo System . . . 16
15.11 IndexedDB Persistence . . . 17

16 Performance Considerations 17

17 Browser Requirements 17

18 Limitations and Known Issues 18

Introduction

Annota is a web-based audio editor inspired by Audacity. It runs entirely in the browser with no server-side processing. The application is built with:

- **TypeScript** (strict mode) with ES module imports/exports
- **Vite** as the build tool and development server
- **HTML5 Canvas API** for all graphical rendering (waveform, spectrogram, timeline, labels)
- **Web Audio API** for audio decoding, playback, and real-time processing
- **IndexedDB** for persistent project storage
- **Web Workers** for non-blocking STFT computation
- **Zero runtime dependencies** — FFT, WAV encoding, color maps, effects, and all UI are implemented from scratch

Quick Start

Annota requires **Node.js 22+** and **npm**.

1. Install dependencies:

```
npm install
```

2. Start the development server:

```
npm run dev
```

3. Open `http://localhost:5173` in a modern browser.
4. Click **Load** or drag an audio file onto the editor.

Note: Opening `index.html` directly via `file://` will not work. ES modules and Web Workers require an HTTP server, which the Vite dev server provides.

Build Commands

Command	Description
`npm run dev`	Start Vite development server with hot reload
`npm run build`	Production build to `dist/` (minified, tree-shaken)
`npm run typecheck`	Run TypeScript type checking without emitting

Table 1: Available npm scripts

Supported audio formats depend on the browser's `decodeAudioData` implementation. Most browsers support WAV, MP3, OGG, FLAC, M4A, AAC, and WebM.

User Interface Overview

The interface is arranged vertically in the following zones:

1. **Menu Bar** (top): File, Edit, Tracks, Effects, Analysis, Help menus.
2. **Toolbar** (46px): Quick-access buttons for file ops, transport, zoom, clipboard, snap, and theme toggle.
3. **Time Ruler** (30px): Adaptive time axis with tick marks and labels.
4. **Tracks Area** (flexible): Vertically stacked, resizable track rows:
   - **Main Audio Track**: Layered canvases showing waveform or spectrogram, selection overlay, and playback cursor. Left panel shows track metadata and controls (volume, pan, mute, solo).
   - **Extra Audio Tracks**: Additional audio files loaded as parallel tracks with independent controls.
   - **Label Track**: Annotation canvas with point and region labels.
5. **Scrollbar**: Horizontal scrollbar for navigation.
6. **Status Bar** (bottom, 28px): Sample rate, channels, duration, selection range, cursor position, zoom level.

All track rows (audio and labels) are vertically resizable by dragging the handle at the bottom edge of each row.

Menu Bar

Menu	Items
File	New Project, Open Project, Save Project, Load Audio, Add Audio Track, Append Audio, Export Audio, Export Labels, Import Labels
Edit	Undo, Redo, Cut, Copy, Paste, Duplicate, Delete, Select All
Tracks	Add Audio Track, Split Channels, Mixdown Export
Effects	Fade In, Fade Out, Normalize, Adjust Gain, Parametric EQ, Compressor, Reverb, Noise Reduction, Speed/Pitch, Saturation, Resample
Analysis	Spectrogram, Mel Filterbank, MFCC, Save Analysis Image
Help	Keyboard Shortcuts, About

Table 2: Menu bar structure

Toolbar Groups

Loading Audio

File Picker

Click **Load** or press `Ctrl+O` to open the system file picker. Select any supported audio file.

Group	Controls
File	Load, Save Project
Edit	Undo, Redo
Clipboard	Cut, Copy, Paste, Delete
Transport	Play, Pause, Stop
Zoom	Zoom In, Zoom Out, Fit, View Mode (Waveform / Spectrogram)
Speed	Playback speed selector (0.5x – 2x)
Snap & Theme	Snap toggle, Dark/Light mode toggle
Time	Current playback time display

Table 3: Toolbar control groups

Drag and Drop

Drag an audio file directly onto the editor. The drop zone accepts audio files as well as label files (`.txt`, `.json`, `.srt`, `.vtt`, `.xml`).

Multiple Tracks

- **Add Audio Track** (Tracks menu): Loads a second audio file as a parallel track displayed below the main track. Each track has independent volume, pan, mute, and solo controls. All tracks are vertically resizable.
- **Append Audio** (File menu): Loads a file and concatenates it to the end of the current audio.
- **Split Channels** (Tracks menu): For stereo files, splits into separate mono tracks for independent editing.

Playback

Transport Controls

- **Play** (Space): Starts playback from the cursor position. If a selection exists, plays only the selected range.
- **Pause** (Space during playback): Pauses at the current position.
- **Stop**: Stops playback and resets position to the cursor.

During playback, a red cursor line sweeps across the waveform at 60 fps. The view auto-scrolls to keep the cursor visible.

Playback Speed

The speed selector in the toolbar allows playback at 0.5x, 0.75x, 1x, 1.25x, 1.5x, or 2x speed via the Web Audio API's `playbackRate` property.

Seeking

- Click on the **time ruler** to jump to that time position.
- Click on the **waveform area** to set the cursor.
- Press **Home** to jump to the beginning, **End** to jump to the end.

Visualization

Waveform

The waveform view shows amplitude over time. For stereo files, both channels are stacked vertically. Rendering uses a hierarchical peak mipmap for instant response at any zoom level.

A **dB scale** overlay appears on the left edge showing amplitude markings at 0, −6, −12, −24, and −48 dB.

Spectrogram

The spectrogram view shows frequency content over time, computed via Short-Time Fourier Transform (STFT):

- FFT size: 2048 samples
- Hop size: 512 samples
- Window: Hann
- Color map: Magma
- Dynamic range: −90 dB to 0 dB

A **frequency scale** overlay appears on the left edge showing frequency markings in Hz/kHz.

Spectrogram computation is delegated to a Web Worker with progress reporting. On browsers without Worker support, it falls back to chunked main-thread computation that yields periodically to avoid freezing the UI.

Zoom and Scroll

- **Ctrl+Scroll**: Zoom in/out centered on the mouse position.
- **Scroll** (no modifier): Horizontal scroll.
- +/− buttons: Step zoom by factor of 2.
- **Fit**: Zoom to show the entire audio file.
- **Arrow keys**: Scroll left/right.

The zoom level is measured in *samples per pixel* (SPP). Minimum SPP is 4 (most zoomed in), maximum is 65536 (most zoomed out). The status bar shows the equivalent pixels-per-second.

Dark / Light Theme

Click the moon icon in the toolbar to toggle between light and dark themes. The theme preference is persisted in `localStorage`. All canvas renderers (waveform, spectrogram, timeline, selection, labels, axis overlays) use theme-aware colors that update immediately on toggle.

Selection and Editing

Creating a Selection

Click and drag on the waveform area to create a time selection. The selected region is highlighted with a semi-transparent overlay with vertical edge markers.

If **Snap** is enabled (checkbox in the toolbar), selection edges are quantized to grid intervals that adapt to the zoom level.

Clipboard Operations

- **Cut** (`Ctrl+X`): Removes the selected audio and places it in the clipboard.
- **Copy** (`Ctrl+C`): Copies the selected audio to the clipboard.
- **Paste** (`Ctrl+V`): Inserts clipboard audio at the cursor position.
- **Duplicate** (`Ctrl+D`): Copies and immediately pastes the selection after the selection end.
- **Delete** (`Delete`): Removes the selected audio without copying.
- **Select All** (`Ctrl+A`): Selects the entire audio duration.

All destructive operations are undoable.

Exporting Audio

Via the File menu or toolbar, a dialog offers:

- **WAV** (lossless): Always available.
- **MP3**: Browser-dependent (requires `MediaRecorder` with audio/mpeg support).
- **Sample rate**: Original or resample to 8000, 16000, 22050, 44100, or 48000 Hz.
- **Mono**: Optionally mix down to mono on export.

Audio Effects

All effects are accessible via the **Effects** menu. Effects operate directly on the audio buffer samples and are undoable. When a selection exists, effects apply only to the selected region; otherwise they apply to the entire file.

Fade In / Fade Out

Applies an amplitude envelope across the selected region. A dialog allows choosing the curve type:

- **Linear**: Straight-line ramp.
- **Exponential**: Quadratic curve (t^2 for fade in, $(1-t)^2$ for fade out).
- **S-Curve**: Smooth sigmoid ($3t^2 - 2t^3$).

Normalize

Peak-normalizes audio to -1 dBFS. Finds the peak amplitude in the target region and applies a uniform gain to bring it to the target level.

Adjust Gain

Opens a dialog to enter a gain value in dB. The gain is applied as a linear multiplier ($10^{dB/20}$). Output is clamped to $[-1, 1]$.

Parametric EQ

A multi-band parametric equalizer with visual frequency response curve:

- Multiple bands, each with frequency (Hz), gain (dB), and Q factor.
- Real-time visual preview of the combined frequency response on a canvas.
- Applies cascaded biquad filters to the audio data.

Compressor / Limiter

Dynamics processing with the following parameters:

Parameter	Range	Default	Unit
Threshold	−60 to 0	−20	dB
Ratio	1:1 to 20:1	4:1	—
Attack	0.1 to 100	10	ms
Release	10 to 1000	100	ms
Knee	0 to 20	6	dB
Makeup Gain	0 to 30	0	dB

Table 4: Compressor parameters

Reverb

Algorithmic reverb with three parameters:

- **Room Size** (0–1): Controls the decay time.
- **Damping** (0–1): High-frequency absorption.
- **Wet/Dry Mix** (0–1): Balance between processed and original signal.

Noise Reduction

Two-step spectral noise gate:

1. **Get Noise Profile**: Select a noise-only region and click to capture the noise spectrum.
2. **Apply**: Select the region to clean. The algorithm subtracts the noise profile from each STFT frame, using a sensitivity parameter (0.5–5.0) to control aggressiveness.

The noise reduction dialog is non-modal, allowing the user to change the selection between steps.

Speed / Pitch Change

Resampling-based speed adjustment (0.25x to 4x). Changing speed also changes pitch proportionally. This modifies the buffer length and sample data.

Saturation

Soft-clipping distortion with a drive parameter (1–10). Applies a hyperbolic tangent waveshaper: $output = \tanh(drive \times input)$.

Resample

Changes the audio sample rate via high-quality interpolation. Available target rates: 8000, 16000, 22050, 44100, 48000 Hz.

Analysis

Analysis features are accessible via the **Analysis** menu. Each analysis opens a floating, draggable panel with a canvas visualization. Analysis requires a selection to define the region to analyze.

Spectrogram Analysis

Computes and displays a detailed spectrogram of the selected region in a floating panel. Uses the same STFT parameters as the main spectrogram view (2048 FFT, 512 hop, Hann window) with the Magma colormap.

The implementation uses the function-based FFT:

```
const win = createWindow(fftSize, 'hann');
const real = new Float32Array(fftSize);
const imag = new Float32Array(fftSize);
for (let f = 0; f < numFrames; f++) {
  // Window and copy frame
  for (let i = 0; i < fftSize; i++) {
    real[i] = mono[offset + i] * win[i];
    imag[i] = 0;
  }
  fft(real, imag);
  // Compute magnitudes in dB ...
}
```

Mel Filterbank

Computes mel-scale filterbank energies of the selected region:

- Configurable number of mel filters (default: 40)
- FFT size and hop size parameters
- Displays a 2D heatmap of filter energies over time

MFCC (Mel-Frequency Cepstral Coefficients)

Computes MFCCs from the mel filterbank output:

- Applies log compression to mel energies
- Applies Discrete Cosine Transform (DCT) to extract cepstral coefficients
- Configurable number of coefficients (default: 13)
- Displays a 2D heatmap of MFCC values over time

Save Analysis as Image

Any open analysis panel can be saved as a PNG image:

- Click the save button (floppy disk icon) in the analysis panel header.
- Alternatively, use **Analysis** > **Save Analysis Image** from the menu.
- The canvas content is exported via `canvas.toDataURL('image/png')` and downloaded with an auto-generated filename based on the analysis type.

Label Track (Annotations)

Label Types

- **Point labels**: A single timestamp, displayed as a vertical dashed line with a circle marker and a text badge.
- **Region labels**: A start–end time range, displayed as a filled rectangle with edge handles and a centered text badge.

Creating Labels

- **Click** on empty space in the label track: Creates a point label at that time.
- **Drag** on empty space in the label track: Creates a region label spanning the drag range.
- **Ctrl+B**: Creates a label at the cursor position, or a region label matching the current selection.

Editing Labels

- **Click** a label to select it.
- **Double-click** a label to edit its text inline.
- **Drag** a label body to move it.
- **Drag** a region edge handle to resize it.
- **Delete/Backspace** with a label selected to remove it.

Categories

Labels can be assigned to categories: *speech*, *noise*, *music*, *silence*, or *other*. The label track header provides a category filter dropdown and a search box to filter labels by text.

Export Formats

Import

Click **Import Labels** and select a label file. The format is auto-detected:

- Files starting with `WEBVTT` are parsed as WebVTT.
- Files with `->` timestamp patterns are parsed as SRT.
- Files starting with `[` or `{` are parsed as JSON.
- All other files are parsed as Audacity tab-separated format.

Format	Extension	Description
Audacity	`.txt`	Tab-separated: `start\tend\ttext`. Compatible with Audacity's label import.
JSON	`.json`	Array of objects with `start`, `end`, `text`, `type`, `category` fields.
SRT	`.srt`	SubRip subtitle format. Point labels get a default 2-second duration.
WebVTT	`.vtt`	Web Video Text Tracks format for HTML5 `<track>` elements.
ELAN XML	`.xml`	Simplified ELAN annotation format with time slots and tier structure.

Table 5: Supported label export formats

Per-Track Controls

Volume

Each track has an independent volume slider (0–1 linear). The main track's volume slider is in the left panel; extra tracks have their own slider.

Pan

Each track has a pan slider (-1 = full left, 0 = center, $+1$ = full right). Panning uses the Web Audio API's `StereoPannerNode`.

Mute and Solo

- **Mute (M)**: Silences the track. The button turns red when active.
- **Solo (S)**: When any track is soloed, only soloed tracks produce output. The button turns accent-colored when active.

The `M` key toggles mute on the main track.

Resizable Track Heights

All track rows (audio and labels) can be vertically resized by dragging the resize handle at the bottom edge of each row. The minimum height is 80px for audio tracks and 50px for the label track.

Mixdown Export

The Mixdown dialog mixes all unmuted (or, if any track is soloed, all soloed) tracks into a single WAV file. Samples are summed and clamped to $[-1, 1]$.

Undo / Redo

Destructive operations capture a full snapshot (audio buffer + labels + cursor position) before execution. Up to 30 undo levels are maintained.

- **Ctrl+Z**: Undo the last destructive action.

- **Ctrl+Shift+Z** or **Ctrl+Y**: Redo.

Operations that push to the undo stack include: cut, paste, duplicate, delete segment, concatenate audio, all effects (fade, normalize, gain, EQ, compressor, reverb, noise reduction, speed change, saturation), and label modifications.

Note: Undo snapshots clone the entire `AudioBuffer` as `Float32Array` channel data. For long recordings, this can consume significant memory.

Project Save and Load

Saving

Press `Ctrl+S` or use the File menu. Enter a project name. The full project state is serialized to IndexedDB:

- Audio buffer data (raw Float32 samples per channel)
- All labels (with categories)
- Viewport state (zoom level, scroll position)
- Cursor position and selection
- View mode
- Extra track data

Auto-Save

After loading audio, an auto-save runs every 30 seconds in the background. The auto-save is stored under a reserved key and can be loaded from the project list.

Loading

Use **File** > **Open Project**. A prompt lists all saved projects (including auto-save if available). The project state is fully restored, including audio, labels, and viewport.

Snap to Grid

Enable the **Snap** checkbox in the toolbar. When enabled:

- Selection edges snap to the nearest grid line.
- Cursor placement snaps when clicking on the timeline.
- The grid interval adapts to the zoom level:

Visible Duration	Snap Interval
$< 1\,\mathrm{s}$	$10\,\mathrm{ms}$
$< 5\,\mathrm{s}$	$50\,\mathrm{ms}$
$< 15\,\mathrm{s}$	$100\,\mathrm{ms}$
$< 60\,\mathrm{s}$	$500\,\mathrm{ms}$
$< 300\,\mathrm{s}$	$1\,\mathrm{s}$
$\geq 300\,\mathrm{s}$	$5\,\mathrm{s}$

Table 6: Snap intervals by zoom level

Keyboard Shortcuts

Technical Architecture

Technology Stack

- **Language**: TypeScript (strict mode, ES2020 target)
- **Build**: Vite 8 with native ES module dev server
- **Module system**: ESNext modules with bundler resolution
- **Type checking**: `tsc -noEmit` (no separate compilation step; Vite handles transpilation)
- **Runtime dependencies**: None (zero `dependencies` in `package.json`)

File Structure

```
Annota/
  index.html              HTML layout, toolbar, dialogs, canvas elements
  styles.css              Light/dark themes via CSS custom properties
  package.json            npm scripts: dev, build, typecheck
  tsconfig.json           TypeScript strict mode configuration
  vite.config.ts          Vite build configuration
  src/
    types.ts              Shared interfaces and type definitions
    utils.ts              Time formatting, color maps, HiDPI canvas
    fft.ts                Radix-2 Cooley-Tukey FFT + window functions
    viewport.ts           Central zoom/scroll state + transforms
    undo.ts               Snapshot-based undo/redo manager
    audio-engine.ts       Web Audio decoding, playback, editing
    waveform.ts           Mipmap peak cache + theme-aware rendering
    spectrogram.ts        STFT computation + rendering
    timeline.ts           Adaptive time ruler with theme colors
    selection.ts          Selection overlay + cursor rendering
    label-track.ts        Annotation data + rendering + I/O
    clipboard.ts          Cut/copy/paste for audio and labels
    resampler.ts          Sample rate conversion
    effects.ts            Core effects (fade, normalize, gain)
    worker.ts             Self-contained Web Worker for STFT
    effects/
      eq.ts               Parametric equalizer (biquad cascade)
      compressor.ts       Dynamics compressor with soft knee
      reverb.ts           Algorithmic reverb
      noise-reduction.ts  Spectral noise gate
```

Shortcut	Action
Space	Play / Pause toggle
Ctrl+O	Open file
Ctrl+S	Save project
Ctrl+Z	Undo
Ctrl+Shift+Z / Ctrl+Y	Redo
Ctrl+X	Cut selection
Ctrl+C	Copy selection
Ctrl+V	Paste at cursor
Ctrl+D	Duplicate selection
Ctrl+A	Select all
Ctrl+B	Add label at cursor/selection
Ctrl+=	Zoom in
Ctrl+−	Zoom out
Ctrl+F	Fit to view
M	Toggle mute on main track
Delete	Delete selected audio or label
Backspace	Delete selected label
Home	Jump to beginning
End	Jump to end
Arrow Left	Scroll left
Arrow Right	Scroll right
Escape	Deselect / close dialog

Table 7: Complete keyboard shortcut reference

```
    time-stretch.ts     Resampling-based speed change
  analysis/
    filterbank.ts       Mel filterbank computation
    mfcc.ts             MFCC extraction (DCT on mel energies)
  ui/
    icons.ts            SVG icon strings
    context-menu.ts     Right-click context menu
    menu-bar.ts         Application menu bar
    theme.ts            Light/dark theme manager
    analysis-panel.ts   Floating analysis panel + save as PNG
  app/
    dom-refs.ts         Typed DOM element references (~90 elements)
    app.ts              Entry point: wires everything together
```

Canvas Architecture

The main track area uses stacked `<canvas>` elements:

1. **Spectrogram canvas**: Pre-rendered STFT image.
2. **Waveform canvas**: Peak-based waveform.
3. **dB / Frequency scale canvas**: Axis overlay (switches based on view mode).
4. **Selection canvas**: Selection highlight region.
5. **Cursor canvas**: Playback cursor line (redrawn at 60 fps).

Only one of waveform or spectrogram is visible at a time. This layering ensures that the frequently-updated cursor does not trigger expensive waveform or spectrogram redraws.

Viewport System

The `Viewport` class is the central coordinator for zoom and scroll state:

- `samplesPerPixel` (SPP): The fundamental zoom unit. Range: 4–65536.
- `scrollSamples`: Horizontal offset in samples.
- `sampleRate`: Audio sample rate for time conversion.

All renderers read from the viewport to determine what portion of the audio is visible. Coordinate transforms:

- `timeToPixel(seconds)` → canvas X position
- `pixelToTime(px)` → time in seconds
- `sampleToPixel(sampleIndex)` → canvas X position

Waveform Mipmap

The waveform renderer pre-computes a peak pyramid (min/max per block) at power-of-2 block sizes from 2 to 65536 samples. Each coarser level is derived from the finer level by comparing pairs:

```
# Pseudocode for mipmap construction
for spp in [2, 4, 8, ..., 65536]:
    if spp == 2:
        compute peaks directly from samples
    else:
        derive from level[spp/2] by comparing pairs
```

This ensures $O(N)$ total construction time and $O(1)$ peak lookup at any zoom level.

STFT and Spectrogram

The spectrogram is computed via Short-Time Fourier Transform:

1. Audio is converted to mono (channel averaging).
2. **A copy** of the mono data is transferred to the Web Worker (the original buffer is preserved).
3. A sliding Hann window (2048 samples, 512-sample hop) extracts frames.
4. Each frame is FFT-transformed using the Radix-2 Cooley-Tukey algorithm.
5. Magnitudes are converted to dB and mapped to colors via the Magma colormap.
6. The result is returned as RGBA pixel data and written to an offscreen canvas.

The visible portion is drawn via `drawImage` with source rectangle clipping, which is GPU-accelerated in most browsers.

FFT Implementation

The FFT module (`fft.ts`) exports a **function-based** API, not a class:

```
// Function signature: operates in-place on real/imag arrays
fft(real: Float32Array, imag: Float32Array): void

// Window function creation
createWindow(size: number, type: WindowType): Float32Array

// Convenience: compute magnitude spectrum
magnitudeSpectrum(real, imag): Float32Array

// Full STFT computation
computeSTFT(samples, fftSize, hopSize, windowType): number[][]
```

Supported window types: `hann`, `hamming`, `blackman`, `bartlett`, `rectangular`.

Web Worker

The `worker.ts` file contains a self-contained copy of the FFT, window functions, and color map. This intentional duplication avoids import issues in the worker context. It handles three message types:

- `computeSTFT`: Full spectrogram computation with progress reporting.
- `computePeakMipmap`: Peak pyramid construction.
- `encodeWAV`: WAV file encoding.

Audio data is copied before being transferred to the worker to prevent detaching the main AudioBuffer's backing memory.

Theme System

The `ThemeManager` class manages light/dark mode:

- Persists preference in `localStorage`.
- Sets `data-theme` attribute on `<html>` for CSS variable switching.
- Provides a `ThemeColors` object with named color values for all canvas renderers.
- On toggle, propagates colors to: WaveformRenderer, SpectrogramRenderer, TimeRuler, SelectionManager, LabelTrack, and all extra track waveform renderers.

CSS custom properties (defined in `:root` and `[data-theme="dark"]`) handle all DOM element styling.

Undo System

The `UndoManager` maintains two stacks (undo and redo). Before each destructive operation, a snapshot is pushed:

- Audio buffer: All channel data is cloned as `Float32Array` objects (stored as plain objects with `sampleRate`, `numberOfChannels`, `length`, and `channels` fields).
- Labels: Deep-cloned via `JSON.parse(JSON.stringify(...))`.

- Cursor position.

On undo, the current state is pushed to the redo stack and the snapshot is restored by reconstructing an `AudioBuffer` via `AudioContext.createBuffer()`.

IndexedDB Persistence

Projects are stored in IndexedDB under database `annota_projects`. Each project entry contains:

- Timestamp
- Audio data (sample rate, channel count, length, raw Float32 channel arrays)
- Labels array (with categories)
- Viewport state
- Cursor position
- View mode
- Extra track data

Auto-save writes to a reserved key (`__autosave__`) every 30 seconds.

Performance Considerations

- **Waveform rendering**: The mipmap ensures $O(W)$ rendering cost where W is the canvas width in pixels, regardless of audio length.
- **Spectrogram**: Computed once, then rendered via `drawImage` (near-zero cost per frame).
- **Selection/cursor**: Drawn on dedicated canvases, avoiding full redraw.
- **Undo memory**: Each snapshot stores the full audio buffer. For a 10-minute stereo 44.1 kHz file, each snapshot is approximately 106 MB. With 30 levels, this can reach 3 GB.
- **Worker offloading**: STFT computation runs in a Web Worker with progress reporting, keeping the UI responsive.
- **Production build**: Vite produces a tree-shaken, minified bundle of approximately 120 KB JS + 15 KB CSS (32 KB + 3 KB gzipped).

Browser Requirements

- **Web Audio API**: Required for audio decoding and playback.
- **Canvas 2D**: Required for all rendering.
- **IndexedDB**: Required for project save/load.
- **Web Workers**: Used for STFT offloading. Falls back to main-thread computation.
- **ES2020+**: Modules, `class`, `async/await`, optional chaining, nullish coalescing.

Tested on Chrome 90+, Firefox 88+, Safari 15+, and Edge 90+.

Limitations and Known Issues

- No recording from microphone (could be added via `MediaRecorder`).
- MP3 export depends on browser `MediaRecorder` support and may fall back to WAV.
- The undo system stores full buffer snapshots, which is memory-intensive for large files. A diff-based or command-pattern undo could reduce memory usage.
- Label track supports only a single tier. Multiple tiers would be useful for complex annotation workflows.
- No spectral editing (painting on the spectrogram to modify audio).
- No real-time effect preview — all effects are applied offline to the buffer.
- Requires Node.js 22+ and `npm install` to run (cannot open `index.html` directly).